

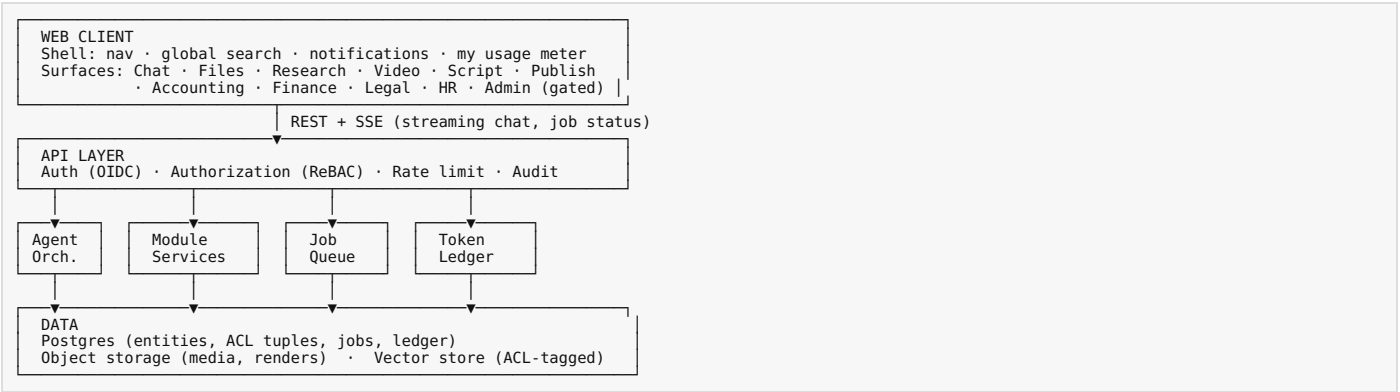
Video Agent Platform — Build Specification

v2.4 · 15 Aug 2026 · Aura Farmers, Inc. · engineering

An internal work platform for a video media company. Each employee has an AI agent scoped to their job, eight modules — four for content production and four for back-office work — and a shared media and document database with per-file permissions. An admin panel controls accounts, module access, file access, AI spend, and a knowledge-and-skills area through which the client tunes agent behaviour without us.

Product UI language: Traditional Chinese, Hong Kong usage. This document is English.

1. Architecture



Suggested stack. Postgres 16, Redis for the queue, S3-compatible object storage, OpenFGA or SpiceDB for authorization, pgvector for retrieval, a Node or Python API, React front end. Substitutions are fine as long as authorization stays in a ReBAC engine — OpenFGA, SpiceDB or equivalent — rather than a hand-rolled table.

2. Authorization

An agent operates with exactly the permissions of the employee who invoked it.

2.1 Two layers

Platform roles (RBAC), four only:

Role	Can
admin	Everything below, plus manage accounts, roles, module entitlements, file-group assignments, budgets, and view all files (logged). Can create other admins
builder	Configure agent prompts and module settings for their team
member	Use their entitled modules; read and write files they have access to
guest	Time-boxed access to named folders only. No module access unless explicitly granted

Content permissions (ReBAC). Zanzibar-style relation tuples in OpenFGA or SpiceDB. Do not hand-roll a `permissions` table; the first "share one folder with a freelancer" request breaks it.

```
folder:2026-Q3-campaign#editor@group:production
folder:2026-Q3-campaign#viewer@user:amy
file:master-004#parent@folder:2026-Q3-campaign
```

Relations: `owner > editor > commenter > viewer`. Folders inherit to children. Broadest grant wins. Groups are subjects, so team membership resolves transitively.

2.2 Enforcement requirements

- Check at query time, not index time.** Every retrieval — file list, search, vector lookup, agent context assembly — filters against the requesting user.
- Tag every embedding chunk** with its resource ID and filter inside the vector query, not after. An unfiltered index leaks documents through search results with no visible error.
- Revocation is immediate.** Maintain a deny-list checked at serve time so a removed grant applies before any reindex.
- Filter citations with the same check as content.** A leaked title discloses that a document exists.
- Shared output uses the least-common-denominator.** If agent output is posted to a group surface, restrict source material to what every member of that group can read.
- Scheduled jobs need their own identity.** A background agent running while the employee is offline gets a scoped service principal, created by an admin and displayed in the UI. It is the only exception to the rule above.
- Sharing is bounded by the sharer.** An employee can grant only relations they hold. Enforce server-side; do not rely on the UI hiding the option.

2.3 Test requirement

Write an adversarial permission suite before the modules. Minimum cases: employee searches for a document they cannot open; agent asked directly for its contents by filename; agent asked to summarise a folder containing mixed-permission files; permission revoked mid-session; guest attempts to reshare; citation list from a mixed-permission retrieval. All must fail closed.

3. Data model

Core tables. Names are indicative; keep the relationships.

Table	Key fields

users	id, email, display_name, role, status, locale, created_at
groups / group_members	team and department membership, used as ReBAC subjects
module_entitlements	user_id, module, enabled_by, enabled_at
folders / files	id, parent_id, name, kind (video , script , report , asset , comment_export), owner_id, storage_key, mime, bytes, checksum, deleted_at
file_versions	file_id, version, storage_key, author_id, created_at, note
acl_tuples	delegated to OpenFGA/SpiceDB; mirror only what you need for listing
jobs	id, type, status, payload, provider, provider_job_id, attempts, cost_cents, requested_by, resource_id, created_at, completed_at
token_ledger	id, user_id, module, model, provider, prompt_tokens, completion_tokens, cost_cents, request_id, occurred_at
budgets	scope (user / team), scope_id, period, cap_cents, spent_cents, action_on_cap
channels	platform, account_ref, credential_ref, scopes, audit_status, quota_state, connected_by
publish_items	file_id, channel_id, state, caption, scheduled_for, approved_by, platform_response, published_at
audit_log	actor_id, action, resource_type, resource_id, before, after, ip, occurred_at
model_configs	scope (tenant / module / role), model, provider, params, system_prompt, version, active, created_by
migrations	requested_by, source, target, state, item_count, bytes, verified_at, report_key
knowledge_files	id, kind (instruction / style / skill / example), scope (global / module / role), scope_id, storage_key, version, active, uploaded_by, updated_at
ledger_documents / ledger_entries	source document, extracted fields, suggested account, confirmed_by, journal lines, export_batch
budgets_fin / spend_requests	budget lines by department and period; spend requests with amount, approver chain, state
contracts / contract_clauses	counterparty, dates, renewal terms, owner, reminder_at; clause deviations from template
leave_requests / leave_balances	employee, type, dates, approver chain, state
requisitions / candidates / applications	job requisition and approval; candidate records; parsed applications with ranking and interview slots

Retention: soft delete on files with a 30-day recovery window, then hard delete. Audit log retained for the life of the engagement.

4. Client surfaces

Every module has its own working screens. Chat is one surface among several; nothing in a module is reachable only through chat.

4.1 Shell

Left navigation shows only entitled modules. Top bar carries global search (permission-filtered), notifications (job completions, approval requests, budget warnings), and a personal usage meter showing the employee's own consumption against their cap. Locale is Traditional Chinese with an English toggle.

4.2 Chat

Streaming responses, tool-call transparency (which module ran, which files were read), inline file previews, and citations that link to the file. A side panel lists the resources the current answer used. Chat can trigger any module the employee is entitled to, and results land in their file space.

4.3 Market Research

Four screens plus chat.

Trends dashboard. Source selector (client-designated sources), region selector defaulting to HK/TW/SG, and a timeframe control. Ranked topic cards, each showing a heat score, a momentum sparkline over the selected window, a one-line fact summary, suggested angles, and links to primary sources. Actions per card: adopt, reject, save for later. Adopt and reject feed back into ranking weights. Sensitive topics carry a flag and a reason.

Search and compare. Keyword and topic search across connected sources, with up to five series on one time axis for comparison. Results exportable to the database as a research report.

Content performance. The company's own published videos. Filters by channel, date range and campaign. Table view with views, watch-through rate, engagement rate and comment volume; time-series view for any selected video; per-channel breakdown; top and bottom performers for the period. Loads without a chat prompt, and defaults to the last 28 days.

Comment inbox. Incoming comments across channels that expose a comment interface, grouped by video. Filters for sentiment, language, flagged keywords and business-lead signals. Each comment shows the AI-suggested reply as a draft; replies require human approval before sending. Bulk actions for hide and mark-as-spam where the platform supports them.

Topic backlog. Adopted topics queued for scripting, with owner, target channel and due date. Hands off directly to the Script module. Chart guidance: time series as line charts, channel comparison as horizontal bars, distribution as a simple histogram. One accent colour plus neutrals; no chart junk; every chart states its date range and source.

4.4 Script

Brief intake. Structured form — topic (or pulled from the backlog), angle, target channel, target duration, tone, mandatory points, things to avoid.

Split editor. Draft on the left, style sidebar on the right showing house-style conformance, flagged terms, estimated spoken duration, and reading level. Inline suggestions accept or reject individually. Natural-language rewrite commands operate on a selection.

Version history. Diff between any two versions, with author and timestamp.

Approve and lock. Locking produces the single authorised version and hands it to the Video module. Locked scripts are read-only; changes create a new version and require re-approval.

House style comes from a prompt template plus retrieval over the client's approved past scripts. No fine-tuning; out of contract scope (A7).

4.5 Video Edit

Project workspace. Locked script on the left, shot list on the right. Each script beat becomes a shot card showing its source (existing

footage, generated clip, b-roll, still), duration, and status.

Media bin. Database files filtered to what the employee can access, with type, duration, resolution and tag filters. Drag onto a shot card to assign.

Render queue. Every generation and render job with state, progress, provider, attempt count and cost. Failed jobs show the provider error and a retry control. Costs roll up to the project.

Preview. Player with subtitle overlay, aspect-ratio switcher (16:9, 9:16, 1:1) and version comparison against the previous cut.

Audio panel. Voice selection, alternate takes, music selection, and a ducking control for background music under voice-over.

Export. Multi-format render to the selected aspect ratios, plus subtitle file, audio file and cover images, archived to the database with the project.

4.6 Publish

Channel board. One card per connected channel showing connection state, granted scopes, audit or verification status, remaining quota, and the last successful publish. Channels that are connected but not yet audited are marked, with the consequence stated on the card.

Composer. Per-channel title, caption, tags, thumbnail and scheduling, with a shared master version and per-channel overrides. Character limits and format rules enforced per platform.

Approval queue. Every item routes to a named approver before transmission. Approve, reject with comment, or request changes. Nothing leaves the platform without an approval record.

Publish log. What went to which channel, when, approved by whom, with the platform response and the resulting URL. Failures show the platform error and whether a retry is safe.

4.7 Accounting

A bookkeeping assistant, not a ledger of record. Output is drafts for a human accountant to confirm; nothing posts anywhere without confirmation.

Document inbox. Upload or forward receipts, invoices and bank statements. OCR and field extraction (date, amount, counterparty, line items, tax), a suggested account from the client's chart of accounts, and a confidence indicator. The user confirms or corrects each document; corrections feed back into future suggestions.

Draft entries. Confirmed documents become draft journal lines. Reconciliation prompts flag bank lines with no matching document and documents with no matching payment.

Period summary. Income and expense by account, receivables and payables ageing, count of unprocessed documents.

Export. Approved entries in the import format of the client's designated accounting software. Build one exporter behind an interface; expect to add formats.

4.8 Finance

Budget. Budget lines by department, project and period against templates the client supplies, with actuals from confirmed accounting entries alongside.

Cash-flow projection. Forward view from client-entered inflows and outflows. Label every projection as indicative.

Cost dashboard. Cost by category over time, including the AI consumption already in the token ledger.

Spend requests. A request with amount, purpose and attachments routes through an approver chain determined by configurable thresholds. Approve, reject, request more information. Full history on each request.

Report drafts. Periodic management report generated from the above for a human to edit.

4.9 Legal

Document drafting. Generate contracts, letters and internal documents from templates the client supplies and approves, with merge fields and a guided intake for the variable parts.

Clause review. Upload an inbound document; the module aligns it against the client's template and configured positions, and marks each departure with a short explanation. Output is a marked-up view plus a summary list — never a verdict.

Contract repository. Signed documents with counterparty, dates, renewal and notice terms, owner. Reminders ahead of key dates.

Compliance checklists. Recurring items the client configures, with owners and due dates.

The module carries a standing disclaimer in the UI that it does not give legal advice. That disclaimer is a contract term (Clause 8.4), so it is not optional.

4.10 Human Resources

Personal data throughout. Every screen here sits behind a dedicated **hr** permission scope; the agent reads HR records only when invoked by someone who holds it.

Leave. Apply, approver routing, balance tracking, team calendar. Leave types and accrual rules are client configuration.

Recruitment. Requisition with approval chain, job description drafting from a role brief, posting to recruitment channels the client holds, intake of applications and résumés submitted to the client, parsing and ranking against the requisition, interview scheduling, structured interview notes.

Candidate records. Past applicants and referrals, searchable, with consent and retention dates. **No sourcing from external**

platforms. The module works only with applications the client receives and channels the client owns; it does not fetch, scrape or enrich profiles from LinkedIn or anywhere else. This is a contract commitment (Schedule A3(8)).

Employee records. Core record, role, contract reference, onboarding and offboarding checklists.

4.11 Admin

People. Table of employees with role, team, status and last active. Row actions to edit entitlements, suspend or remove.

Entitlements matrix. Employee against modules and file groups, editable in place. Top-level admin screen, not nested under settings — this is the onboarding path.

Token dashboard. Consumption over time as a line chart, split by module. A by-employee table with request count, prompt and completion tokens, cost and trend. A by-model breakdown. Filters for person, team and date range. CSV export. Figures update in near real time, with a stated lag if aggregation is batched.

Budgets. Caps per employee and per team. On reaching a cap the platform stops further consumption and notifies the admin and the employee.

Channels and credentials. Connection status, scope inventory, audit state, key rotation dates. Keys are referenced, never displayed.

Audit log. Filterable by actor, action, resource and date. Admin file access appears here like any other access.

Knowledge and skills area. The client's own tuning surface, and a contract deliverable (Schedule A2(a)). Admins upload Markdown files of four kinds — instructions, style guides, skills (step-by-step procedures with examples for a specific task), and worked examples — and scope each file to the whole tenant, a module, or an employee role. Files are versioned with diff and rollback, can be toggled active/inactive, and every change lands in the audit log. At runtime the active files for a given agent are assembled into its system prompt (instructions and style, in a fixed order with a size budget) and into its retrieval corpus (examples and skills, embedded and ACL-tagged like any other file). Show a preview of the assembled system prompt so an admin can see what the agent will actually receive. This area replaces most of our post-handover tuning work, so it has to be usable by a non-engineer: plain upload, plain toggle, plain rollback.

5. Assistant agent

One agent per employee, configured for their job. It holds the conversation, decides which module to call, and writes results into the employee's file space.

Model access goes through an account the client selects and holds — by default an aggregator such as OpenRouter, which is what the rest of this section assumes. One key reaches Claude, GPT, Gemini, DeepSeek, Qwen, Kimi and GLM, so the client switches models from the admin panel without a deploy. Base URL `https://openrouter.ai/api/v1`, bearer auth, OpenAI chat-completions compatible.

Set `require_parameters: true`. OpenRouter silently drops unsupported parameters by default, so a `response_format` or tool spec can vanish with no error.

Also build a direct adapter for any OpenAI-compatible provider, so the client can connect a provider directly or bring their own key rather than routing through OpenRouter. Anthropic's messages format needs a second small adapter. Base URL, model name, key reference and generation parameters are all configuration rows, not constants.

Pin providers where it matters. OpenRouter's default routing moves between providers, which breaks prompt caching and varies latency. For cache-sensitive or latency-sensitive paths, set `order`, `allow_fallbacks: false` and a stable `session_id`.

Turn on zero-data-retention at account level and per request (`"provider": {"zdr": true}`), and record it in the handover documentation. It does not cover plugins such as web search. Keep `:free` model variants off any path touching client data — they carry separate training-policy settings.

Territory note. OpenAI and Anthropic block Hong Kong billing for direct signup; Gemini serves HK natively. Those models are reachable through OpenRouter, but whether HK signup is permitted under OpenRouter's own terms is unconfirmed. Prefer Gemini, DeepSeek, Qwen, Kimi or GLM for anything load-bearing until the client confirms their account.

Chinese output. Mainland-hosted models produce Simplified Chinese and mainland idiom regardless of the system prompt. Run `OpenCC_s2hk` on their output and keep a human review step for anything client-facing.

Every call is metered. Log user, module, model, provider, prompt and completion tokens, cost and request ID to the token ledger at the point of the call, not in a batch job. OpenRouter returns per-request cost in `usage` — use it rather than recomputing.

Handle exhausted credit gracefully. The client funds and manages their own provider accounts, with no contractual minimum balance and nothing for us to monitor. When a call fails because credit or quota has run out, or a subscription has lapsed, show a clear message naming the account and provider, stop retrying, and surface it on the admin panel's credential status screen. It is not a platform fault and it is not ours to fix — the client tops up and it resumes.

5.1 Model tuning

In scope under Schedule A4A, in four layers:

- Prompts and parameters** per module and per employee role, stored as versioned configuration so they change without a deploy.
- Retrieval corpus** built from the client's approved past scripts and materials, ACL-tagged like everything else.
- Model selection** — a harness that runs the client's own examples across several models and records the comparison. The output is handed to the client, so make it reproducible.
- Fine-tuning** where the client's provider supports it. Prepare the training data, run the job **inside the client's own provider account**, and serve the result through that account.

OpenRouter has no fine-tuning API. Tuning happens at OpenAI, Vertex or Bedrock, and the tuned model is served either through OpenRouter BYOK with the client's provider key or through the direct adapter. Build the tuning configuration so a non-engineer can compare two prompt versions and roll back — the client will iterate on this after handover.

6. Job pipeline

Every generation, transcription and render is asynchronous. Build this before the first module.

One `jobs` table, one worker pool, one webhook receiver. Job states: `queued` → `running` → `succeeded` | `failed` | `cancelled`. Exponential backoff with a capped retry count; permanent failures surface to the requesting user with the provider error intact. Idempotency keys on submission so a retried request does not double-charge.

Provider outputs are copied to our own object storage on completion. Some providers expire their CDN links within days; do not treat a provider URL as durable storage.

Cost is recorded per job at completion and rolled up to the project and the token ledger.

7. Integrations

Endpoints, auth and limits are in `02_Integration-Reference.md`. Defaults:

Function	Default	Notes
LLM	OpenRouter, client-owned account	Direct provider adapter as the alternative path
Video generation	Veo 3.1 Fast/Lite via Vertex	Native synchronised audio
Music	ElevenLabs Music	Commercial use cleared. Suno has no official API and its terms prohibit the third-party wrappers, so it stays out of the pipeline
Voice	Azure Speech zh-HK for Cantonese, ElevenLabs for Mandarin and English	Voice clones are registered under the client's own account, never held by us

Transcription	OpenAI <code>gpt-transcribe</code> , Azure for Cantonese	
Assembly	FFmpeg in a queue worker	Shotstack is the fallback if this overruns
Trends	GDELT, YouTube <code>mostPopular</code> , Google Trends API	GDELT permits commercial use. No scrapers, including pytrends

Publishing channels: YouTube, Instagram and Facebook, LinkedIn, WeChat Official Account, TikTok, X. Xiaohongshu, WeChat Channels and Bilibili publish no official interface; those get an exported asset pack for manual posting. Douyin requires a mainland China entity. YouTube and TikTok both require a client-passed compliance audit before content can be public — until then, ship YouTube in draft mode and TikTok via upload-to-inbox.

Comment sync covers YouTube, Instagram and WeChat Official Account. TikTok exposes no comment interface at all.

Weibo was checked and does not work. Its one open write endpoint takes text and an image only and requires a link to a registered domain; video upload is a non-public interface granted case by case to verified institutional accounts; its comment endpoint returns only comments from users who authorised the app; and developer registration needs a mainland business licence, with an ICP filing for website applications. Reasoning in `02_Integration-Reference.md §4.1`.

No scraping, no unofficial interfaces, no circumventing platform protections (Schedule A3(1)).

7.1 Database migration

Migrating the client's data to a different database, storage service or host is a free service under Clause 11.3, available on request at any time, including after the engagement ends. Design for it rather than treating it as a one-off script:

- Keep an export path that produces the full corpus — files, versions, metadata, ACL tuples, audit log — in an open format, and keep it working. It is the same path used at termination.
- Storage access goes through one interface with the backend behind configuration. No storage-provider SDK calls scattered through module code.
- Migration runs as a tracked job with a verification pass that checks item counts, byte counts and checksums, and emits a written report for the client.
- Assume the target may be operated by someone else. Do not require our tooling on the far side.

8. Non-functional requirements

Area	Requirement
Auth	OIDC with the client's identity provider where one exists; email and password with TOTP otherwise. Sessions expire; refresh tokens rotate
Secrets	API keys in a secret manager, referenced by ID. Never in the database in plaintext, never returned to the client, never logged
Transport	TLS everywhere. Signed, expiring URLs for media access
Availability	Single-region is acceptable at this scale. Daily backup with 30-day retention. Restore tested and the test recorded before handover
Performance	File list and search under 500 ms at the 95th percentile for a 100k-file workspace. Chat first token under 2 s. Dashboards under 2 s using pre-aggregated tables
Observability	Structured logs with request IDs, error tracking, and a dashboard for job queue depth, failure rate and provider latency
Rate limits	Per-user and per-tenant limits on agent calls and job submissions, independent of the budget caps
Data handling	Comment data contains personal information. Minimise what is stored, honour deletions, respect platform cache windows. HK PDPO applies
Localisation	All UI strings externalised. Traditional Chinese is the default locale
Browser support	Current Chrome, Edge, Safari and Firefox. Desktop-first; no mobile layout in scope

9. Acceptance criteria

Delivery is accepted against Schedule A of the agreement. Map each item to a demonstrable test before the handover session.

- Admin creates an employee, grants two modules and one file group; that employee sees exactly those modules and files, and no others.
- Admin opens a file they were not explicitly granted; the access appears in the audit log.
- Employee asks their agent about a document they cannot access; the agent does not return, reference or cite it.
- Employee shares a folder they hold `editor` on; the recipient gets no more than `editor`. Attempting to share a folder they only view fails.
- Token dashboard shows consumption by employee, module and model for a given date range, and exports to CSV matching the on-screen figures.
- A team budget cap is reached; further consumption stops and both admin and employee are notified.
- Research dashboard returns ranked topics for a selected source and region, and the performance screen shows metrics for a published video.
- Comment inbox lists comments for a video on a supported channel; a reply sends only after approval.
- Script module produces a draft from a brief, flags style deviations, and locks to a single authorised version.
- Video module produces a rough cut from a locked script and renders 16:9, 9:16 and 1:1 with subtitles.
- Publish module transmits an approved item to a connected channel in draft mode and records the platform response.
- Publish module transmits an approved item to X and records the platform response.
- Admin uploads a style file scoped to the Script module; the next script draft observes it, and the assembled-prompt preview shows the file. Rolling the file back reverts the behaviour.
- Accounting inbox extracts fields from a receipt, suggests an account, and the confirmed entry appears in the export file in the designated format.
- Finance spend request above a threshold routes to the correct approver and cannot be approved by the requester.
- Legal clause review marks the departures between an uploaded document and the client's template.
- HR leave request routes to the configured approver and updates the balance; an employee without the `hr` scope cannot see any HR record, directly or through their agent.
- Permission suite from §2.3 passes in full, including the HR scope cases.
- Documentation delivered; source and configuration repository handed over; training session completed.

10. Build order

1. Auth, roles, and the ReBAC layer with the adversarial permission suite.
2. Database, file browser, sharing UI, versioning, audit log.
3. Job queue, webhook receiver, token ledger.
4. Assistant agent and the two model adapters.
5. Script module — no external dependency beyond the LLM, fastest to demo.
6. Market Research — GDELT and YouTube first, then comment sync.
7. Video Edit — FFmpeg worker before generation.
8. Publish — YouTube and Instagram in draft mode while audits are pending; X once the client's developer account is live.
9. Knowledge and skills area — needed before the tuning work in §5.1 can be handed to the client.
10. HR — first of the back-office modules, because leave is the one people use daily and it exercises the approval-chain component every other back-office module reuses.
11. Finance and Accounting — share the approval chain and the document extraction pipeline.
12. Legal — template drafting reuses the Script module's generation path; clause review is the only new piece.
13. Admin token dashboard, reading the ledger built at step 3.

Steps 1 to 3 are shared plumbing used by every module. Build them first.

11. Client dependencies

The eight-week clock starts at the later of the effective date and receipt of all of these. Log the arrival date of each, in writing.

The client's model access account and key · video, music, voice and transcription keys · X developer account · chart of accounts, budget templates and approval thresholds · contract templates and archive · leave policy, approval hierarchy and employee records · existing style guides and procedures for the knowledge area · developer applications, OAuth credentials and completed platform audits for each channel, registered in the client's name · stock footage licences · brand assets, house style guidelines and approved past scripts · employee list with roles, module entitlements and file groups · a designated administrator and a designated publication approver.

Platform developer registrations and audits take weeks. Start them in parallel with development.